Cours : Cookies & Sessions en PHP

Suite du Cours PHP.

Objectifs : comprendre comment un site « se souvient » d'un visiteur, utiliser correctement cookies et sessions, et appliquer les règles de sécurité associées (authentification, « se souvenir de moi », expiration).

Prérequis : formulaires GET/POST et `htmlspecialchars()` (sections 9 et 13 du Cours PHP), notions de sessions vues à la section 11.

Ressources associées : exercices dans `02_Exercices/Exercices_Sessions_Cookies.pdf` · TP3 Authentification (et ses extensions) dans `03_TP/TP3_Authentification_Sessions_Cookies.pdf`.

---

0. Rappel : HTTP n'a pas de mémoire

Chaque requête HTTP est **indépendante** : le serveur ne sait pas que deux requêtes viennent du même visiteur. Pour « se souvenir », le serveur demande au navigateur de stocker une valeur, que le navigateur **renvoie automatiquement** à chaque requête vers ce site.

```
Navigateur                                  Serveur PHP
    | -- GET /index.php ----------------------> |
    | <-- 200 OK                                |
    |     Set-Cookie: theme=sombre; Max-Age=3600|   ← le serveur demande de stocker
    |                                           |
    | -- GET /index.php ----------------------> |
    |     Cookie: theme=sombre                  |   ← le navigateur renvoie la valeur
```

> **Observer les échanges** : dans les outils de développement du navigateur (F12 → onglet *Réseau*, puis *Application/Stockage* → *Cookies*), ou en ligne de commande avec `curl -i http://localhost/page.php`, on voit les en-têtes `Set-Cookie` et `Cookie`.

---

1. Les cookies

Qu'est-ce qu'un cookie ?

Un cookie est une **petite donnée stockée dans le navigateur** de l'utilisateur. Le serveur demande au navigateur d'enregistrer une valeur, qui est renvoyée à chaque requête vers le même site tant que le cookie n'a pas expiré.

Les cookies permettent par exemple de :

- retenir un thème (clair/sombre) ;
- retenir un pseudo ;
- enregistrer des préférences (langue, affichage) ;
- identifier une session (`PHPSESSID`).

Créer un cookie

```php
setcookie("nom", "Marc", time() + 3600, "/");
```

- `"nom"` → nom du cookie ;
- `"Marc"` → valeur (toujours une **chaîne**) ;
- `time() + 3600` → date d'expiration : dans 1 heure. Sans date, le cookie disparaît à la fermeture du navigateur (*cookie de session*) ;
- `"/"` → chemin : le cookie est valable pour **tout le site**. Sans chemin, il ne concerne que le dossier de la page qui l'a créé (par exemple `/tp/`), et il faudra le supprimer avec ce même chemin.

> **IMPORTANT :** `setcookie()` **doit être appelé avant tout affichage** (HTML, espace, `echo`, y compris le contenu d'un `include` comme `header.php`), car le cookie est envoyé dans les **en-têtes** de la réponse. Sinon : `Warning: Cannot modify header information - headers already sent`.

> Sur XAMPP, `output_buffering` est activé par défaut : l'erreur peut ne pas apparaître sur de petites pages, puis surgir sur un autre serveur. Il faut donc respecter la règle **dans tous les cas**.

Forme recommandée : le tableau d'options (PHP ≥ 7.3)

```php
setcookie("theme", "sombre", [
    "expires"  => time() + 3600 * 24 * 30,
    "path"     => "/",
    "secure"   => false,    // true en production : cookie envoyé en HTTPS uniquement
    "httponly" => true,     // inaccessible depuis JavaScript
    "samesite" => "Lax",    // limite l'envoi lors de requêtes venant d'autres sites
]);
```

Option	Rôle
`expires`	Date d'expiration (horodatage Unix). Absent = disparaît à la fermeture du navigateur
`path`	Chemins du site pour lesquels le cookie est envoyé (`"/"` = tout le site)
`domain`	Domaine concerné (par défaut : le domaine courant)
`secure`	Le cookie n'est envoyé qu'en **HTTPS**. À activer en production (sur `http://localhost` en développement, laisser `false`)
`httponly`	Le cookie est **invisible pour JavaScript** : limite le vol par une faille XSS
`samesite`	`Lax` ou `Strict` : limite l'envoi du cookie depuis d'autres sites (protection **partielle** contre le CSRF)

Lire un cookie

```php
$nom = $_COOKIE["nom"] ?? "";
echo htmlspecialchars($nom, ENT_QUOTES, "UTF-8");
```

Points importants :

- Un cookie qui vient d'être créé avec `setcookie()` **n'apparaît pas** dans `$_COOKIE` pendant la même requête : il sera disponible à la **requête suivante**.
- Le cookie n'existe peut-être pas : utiliser `??` (sinon `Undefined array key`).
- **Un cookie est une donnée envoyée par le client** : l'utilisateur peut la lire et la modifier (outils de développement, `curl`…). Il faut la **valider** et l'**échapper** à l'affichage comme n'importe quelle saisie.

Supprimer un cookie

On le renvoie avec une date d'expiration **passée**, avec les **mêmes** `path` et `domain` qu'à la création :

```php
setcookie("nom", "", ["expires" => time() - 3600, "path" => "/"]);
```

Limites

- environ 4 Ko par cookie, nombre limité de cookies par site ;
- envoyé à **chaque requête** (il alourdit les échanges) ;
- jamais de donnée sensible dans un cookie (mot de passe, prix, droits…).

Exemple (page : `cookie.php`)

```php
<?php
setcookie("theme", "sombre", [
    "expires"  => time() + 3600,
    "path"     => "/",
    "httponly" => true,
    "samesite" => "Lax",
]);
?>
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Cookie</title>
</head>
<body>
    <p>Cookie créé ! Rechargez la page pour le lire dans <code>$_COOKIE</code>.</p>
    <p>Valeur lue : <?= htmlspecialchars($_COOKIE["theme"] ?? "aucune", ENT_QUOTES,
"UTF-8") ?></p>
</body>
</html>
```

---

2. Les sessions

Contrairement aux cookies, les **données d'une session sont stockées sur le serveur**. Le navigateur ne reçoit qu'un identifiant aléatoire, dans un cookie automatique nommé `PHPSESSID`. Cet identifiant permet à PHP de retrouver les données de ce visiteur.

Démarrer une session

```php
session_start();
```

Doit être appelé **avant tout affichage**, comme `setcookie()` (idéalement dès la première ligne, ou dans un fichier `auth.php` inclus en premier).

Version recommandée, avec options de sécurité :

```php
session_start([
    "cookie_httponly" => true,
    "cookie_samesite" => "Lax",
    "use_strict_mode" => true,   // refuse un identifiant de session inconnu du serveur
]);
```

Stocker, lire et supprimer des données

```php
session_start();
$_SESSION["pseudo"] = "Lucas";                          // stocker
echo htmlspecialchars($_SESSION["pseudo"] ?? "", ENT_QUOTES, "UTF-8");   // lire
unset($_SESSION["pseudo"]);                             // supprimer une donnée
```

Détruire complètement la session (déconnexion)

`session_destroy()` **seule ne suffit pas** : elle supprime les données côté serveur, mais `$_SESSION` reste rempli jusqu'à la fin du script et le cookie `PHPSESSID` reste dans le navigateur. Déconnexion complète :

```php
<?php
session_start();
$_SESSION = [];                                         // vide les données du script en cours
setcookie(session_name(), "", ["expires" => time() - 3600, "path" => "/"]);  // supprime le cookie
session_destroy();                                      // détruit la session côté serveur
header("Location: index.php");
exit;
```

Sécurité des sessions

Risque	Parade
Fixation de session : l'attaquant impose un identifiant connu à la victime	`session_regenerate_id(true)` **dès la connexion** ; `use_strict_mode`
Vol de l'identifiant (XSS, écoute du réseau)	cookie `httponly`, `secure` + HTTPS, échapper tous les affichages
Session qui reste ouverte indéfiniment	Expiration par inactivité (voir exercice 5)
Données sensibles	Rester sur le serveur ; ne stocker que le nécessaire

Durée de vie d'une session

- Le cookie `PHPSESSID` est un cookie *de session* : il disparaît normalement à la fermeture du navigateur (certains navigateurs restaurent toutefois les cookies de session au redémarrage).

- Côté serveur, les données sont supprimées par le « ramasse-miettes » après une période d'inactivité (`session.gc_maxlifetime` , 1440 secondes soit 24 minutes par défaut — le nettoyage n'étant pas instantané).

Une session ne dure donc **ni « tant que le navigateur est ouvert » ni indéfiniment** : pour une durée précise, il faut la gérer soi-même (exercice 5).

---

3. Cookies & sessions

Cookies	Sessions
Stockés dans le navigateur	Données stockées sur le serveur (le navigateur n'a que l'identifiant)
Durée définie par `expires` (sans date : jusqu'à la fermeture du navigateur)	Durée limitée par l'inactivité (`gc_maxlifetime`) et par la durée du cookie `PHPSESSID`
Modifiables par l'utilisateur : aucune confiance	Non modifiables par l'utilisateur ; l'identifiant reste à protéger
Taille limitée (≈ 4 Ko)	Peu limitées en pratique
Pas de `session_start()`	Nécessite `session_start()`

- Pour l'**authentification**, le **panier** et toute donnée dont le serveur doit être sûr → **sessions** (ou base de données).
- Pour retenir un **thème**, une **langue** ou un pseudo (données non sensibles) → **cookies**.

> Un panier stocké dans un cookie peut être modifié par l'utilisateur (quantités, prix). Même en session, le **prix est toujours relu en base** côté serveur (voir TP2 Boutique).

---

4. Exemple : pseudo retenu avec un cookie

`login.php`

```php
<?php
$pseudo = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $pseudo = trim($_POST["pseudo"] ?? "");

    if ($pseudo !== "" && mb_strlen($pseudo) <= 30) {
        setcookie("pseudo", $pseudo, [
            "expires"  => time() + 3600 * 24 * 7,
            "path"     => "/",
            "httponly" => true,
            "samesite" => "Lax",
        ]);
    }
} elseif (is_string($_COOKIE["pseudo"] ?? null)) {
    $pseudo = $_COOKIE["pseudo"];          // première visite après le cookie : on préremplit
}
?>
<form method="post">
    <label for="pseudo">Pseudo :</label>
    <input type="text" id="pseudo" name="pseudo"
           value="<?= htmlspecialchars($pseudo, ENT_QUOTES, "UTF-8") ?>">
    <button type="submit">OK</button>
</form>
```

→ Le champ pseudo est **prérempli automatiquement** grâce au cookie.

Points d'attention : la valeur est **échappée** (un cookie peut contenir `"><script>…`), le cookie est créé avant tout affichage, et le champ affiche la valeur qui vient d'être envoyée (elle n'est pas encore dans `$_COOKIE`).

---

5. Exemple : authentification avec session

```php
<?php
session_start(["cookie_httponly" => true, "cookie_samesite" => "Lax", "use_strict_mode" =>
true]);

// Compte de test : le mot de passe est stocké sous forme de hash
// (hash obtenu une fois avec : echo password_hash("1234", PASSWORD_DEFAULT);)
$compteTest = [
    "login" => "admin",
    "hash"  => '$2y$10$Ek7Ngq5uxOj.X7c2JvRVB.VNwGHZQliqNZPX4YkIX6bGScTyTQyRu',
];

$erreur = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $login = trim($_POST["login"] ?? "");
    $pass  = $_POST["password"] ??
"";   // champ « password », comme dans l'exercice 3 et le TP3

    if ($login === $compteTest["login"] && password_verify($pass, $compteTest["hash"])) {
        session_regenerate_id(true);       // protège contre la fixation de session
        $_SESSION["user"] = $login;
        header("Location: private.php");
        exit;
    }
    $erreur = "Identifiants incorrects.";
}
?>
```

Sur `private.php` :

```php
<?php
session_start(["cookie_httponly" => true, "cookie_samesite" => "Lax", "use_strict_mode" =>
true]);

if (!isset($_SESSION["user"])) {
    http_response_code(403);
    echo "Accès interdit.";
    exit;
}
echo "Bienvenue : " . htmlspecialchars($_SESSION["user"], ENT_QUOTES, "UTF-8");
```

> Un mot de passe écrit dans le code (même haché) reste un **compte de test**. Un vrai site stocke les comptes dans une base de données (section 12 du Cours PHP).

---

6. « Se souvenir de moi » : ce qu'il ne faut pas faire

Version **dangereuse** :

```php
setcookie("remember", $pseudo, time() + 3600 * 24 * 7);   // ❌
// puis : si le cookie "remember" existe → l'utilisateur est considéré comme connecté
```

Un cookie est modifiable par le client : il suffit de créer à la main un cookie `remember=admin` pour **être connecté en tant qu'administrateur, sans mot de passe**. (L'extension E1 du TP3 le démontre.)

Principe correct : le cookie contient un **jeton aléatoire** impossible à deviner, dont le serveur conserve l'**empreinte** :

1. À la connexion (case cochée) : `$token = bin2hex(random_bytes(32));`
2. Le serveur stocke `hash("sha256", $token)` avec l'utilisateur ; le cookie contient `pseudo:token` (`httponly`, `samesite`).
3. À la visite suivante, sans session : on lit le cookie, on retrouve l'utilisateur, on compare avec `hash_equals()`. Si c'est bon → on ouvre une session (`session_regenerate_id(true)`).
4. À la déconnexion : on **supprime le jeton côté serveur** et le cookie.

Ainsi, même si la base ou le fichier de comptes fuite, les jetons ne sont pas utilisables directement, et un jeton peut être révoqué.

---

7. Schémas de fonctionnement

Sessions

```
 Navigateur                              Serveur
_____________                          _____________
 | 1. POST login/mot de passe  ————————> | Vérifie les identifiants
 |                                       | session_regenerate_id(true)
 |                                       | $_SESSION["user"] = ...  (stocké côté serveur)
 | <———— 2. Set-Cookie: PHPSESSID=abc —— | Renvoie l'identifiant
 | 3. Chaque requête :                   |
 |    Cookie: PHPSESSID=abc  ——————————> | PHP retrouve la session grâce à l'identifiant
```

Cookies

```
Navigateur  <—— Set-Cookie: theme=sombre ——  Serveur     (setcookie())
Navigateur  —— Cookie: theme=sombre ————————>  Serveur     ($_COOKIE["theme"] à la requête
suivante)
```

Exercices et TP : `02_Exercices/Exercices_Sessions_Cookies.pdf` (exercices 1 à 5), puis `03_TP/TP3_Authentification_Sessions_Cookies.pdf` (TP et extensions E1 à E6).